

EN UNA FRASE Sabes dejar la API lista para producción: expones salud, información y métricas con Actuator (protegidas con Spring Security), separas la configuración con el perfil `prod` y variables de entorno, y levantas la API junto a PostgreSQL con un `Dockerfile` multicapa y un solo `docker compose up`.

LO ESENCIAL

<code>spring-boot-starter-actuator</code>	Añade endpoints de operaciones bajo <code>/actuator</code> ; por defecto solo <code>health</code> e <code>info</code> se exponen sobre HTTP
<code>/actuator/health</code>	Estado de la aplicación y sus dependencias; devuelve 503 si la base de datos cae. Lo usan balanceadores, Kubernetes y el <code>HEALTHCHECK</code> de Docker
<code>management.endpoints.web.exposure.include</code>	Lista de endpoints expuestos; en el proyecto, <code>health,info,metrics</code>
<code>management.endpoint.health.show-details=when-authorized</code>	El detalle de cada componente (<code>db</code> , <code>diskSpace</code>) solo se muestra a usuarios autorizados
<code>info.app.*</code>	Propiedades que devuelve <code>/actuator/info</code> (nombre, versión, descripción)
Perfiles dev / prod	<code>application-dev.properties</code> (H2) y <code>application-prod.properties</code> (PostgreSQL, valores por variable de entorno)
<code>SPRING_PROFILES_ACTIVE</code>	Variable de entorno que elige el perfil; sin ella manda <code>spring.profiles.active=dev</code>
Imagen / contenedor	Plantilla inmutable con todo lo necesario / instancia en ejecución de esa imagen
Dockerfile en tres etapas	Compila con Maven, extrae las capas con <code>-DjarMode=layertools</code> y monta la imagen final con usuario sin privilegios
Capas del JAR	<code>dependencies</code> , <code>spring-boot-loader</code> , <code>snapshot-dependencies</code> y <code>application</code> : se copian de menor a mayor frecuencia de cambio para aprovechar la caché
<code>docker-compose.yml</code>	Define la API y PostgreSQL como un sistema; <code>db</code> es el hostname de la base de datos dentro de la red de Compose
<code>depends_on + condition: service_healthy</code>	La API espera a que PostgreSQL pase su <code>healthcheck</code> (<code>pg_isready</code>)
Volumen nombrado <code>datos_biblioteca</code>	Conserva los datos de PostgreSQL aunque se eliminen los contenedores
<code>-e / environment:</code>	Injectan configuración (<code>DB_URL</code> , <code>DB_USUARIO</code> , <code>DB_PASSWORD</code> , <code>PORT</code>) sin modificar la imagen

ERRORES TÍPICOS

1 `/actuator/env` enseña secretos y contraseñas a cualquiera

- ✓ no uses `management.endpoints.web.exposure.include=*`. Expón solo `health,info,metrics` y protege el resto con `hasRole("ADMIN")`.

2 `SQLException: Connection to db:5432 refused` al hacer `docker compose up`

- ✓ `depends_on` a secas no espera a PostgreSQL. Usa `condition: service_healthy` y un `healthcheck` con `pg_isready` en `db`.

3 La base de datos aparece vacía tras `docker compose down` y `up`

- ✓ declara un volumen nombrado (`datos_biblioteca:/var/lib/postgresql/data`). Solo `down -v` borra los volúmenes.

4 Cada `docker build` vuelve a descargar todas las dependencias

- ✓ copia `pom.xml` y ejecuta `mvn -B dependency:go-offline` antes de `COPY src/ src/`.

5 Los tests intentan conectarse a `localhost:5432`

- ✓ tienes `SPRING_PROFILES_ACTIVE=prod` definida en el sistema o en el IDE. Anota el test con `@ActiveProfiles("dev")` o crea un perfil `test` con H2.

EN EL PROYECTO BIBLIOTECA

El `pom.xml` solo suma `spring-boot-starter-actuator`; el driver de PostgreSQL está desde el Capítulo 9. `SeguridadConfig` añade las reglas de `/actuator/**`, `application.properties` las propiedades `management.*` e `info.app.*`, y `application-prod.properties` pasa a leer `DB_URL` y `DB_USUARIO` del entorno. En la raíz aparecen `Dockerfile` y `docker-compose.yml`. Las migraciones V1 a V3 sirven tal cual en PostgreSQL porque usan SQL estándar; un script ya aplicado no se reescribe, los cambios van en uno nuevo. Con esto el proyecto del volumen queda completo.

■ CÓDIGO MÍNIMO Y PETICIONES

Las dos reglas nuevas de `SeguridadConfig`, antes de las de `/libros`: salud e info públicas, el resto de Actuator solo para `ADMIN`.

```
.requestMatchers("/actuator/health", "/actuator/info").permitAll()
.requestMatchers("/actuator/**").hasRole("ADMIN")
```

JAVA

La conexión en `application-prod.properties`: `${VARIABLE:valor-por-defecto}`.

```
server.port=${PORT:8080}
spring.datasource.url=${DB_URL:jdbc:postgresql://localhost:5432/biblioteca}
spring.datasource.username=${DB_USUARIO:biblioteca_user}
spring.datasource.password=${DB_PASSWORD}
spring.datasource.driver-class-name=org.postgresql.Driver
```

PROPERTIES

El servicio `api` de `docker-compose.yml` (el servicio `db` usa `postgres:16-alpine` y no publica el puerto 5432 al host):

```
api:
  build: .
  depends_on:
    db:
      condition: service_healthy
  environment:
    SPRING_PROFILES_ACTIVE: prod
    DB_URL: jdbc:postgresql://db:5432/biblioteca
    DB_USUARIO: biblioteca_user
    DB_PASSWORD: ${DB_PASSWORD:-cambia-esta-clave}
    PORT: 8080
  ports:
    - "8080:8080"
```

YAML

```
docker build -t biblioteca-api:1.0.0 .      # construye la imagen (compila el JAR dentro)
docker compose up -d --build                # API + PostgreSQL en segundo plano
docker compose ps                           # ¿están los dos contenedores UP?
docker compose logs -f api                  # logs solo de la API
docker compose down                         # elimina contenedores; los volúmenes persisten
docker compose down -v                      # elimina también los volúmenes (borra la BD)
```

TERMINAL

```
curl http://localhost:8080/actuator/health # público, sin token
curl -X POST http://localhost:8080/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"admin123"}'
curl http://localhost:8080/actuator/metrics \
  -H "Authorization: Bearer $TOKEN"         # exige token de admin
```